

Project Management Report

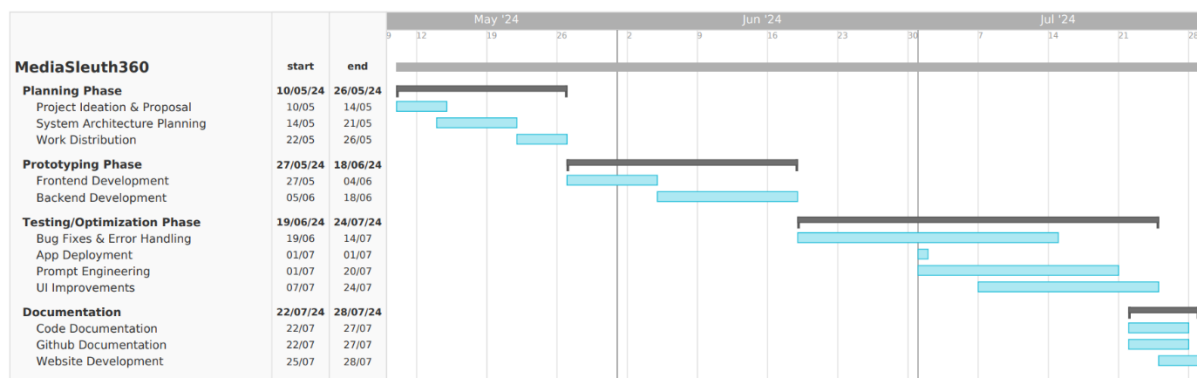
Project Vision

MediaSleuth360 aims to revolutionize how users interact with media content by leveraging advanced AI technologies. The platform can be the go-to tool for extracting, understanding, and navigating information from video and audio files, enhancing productivity and accessibility for users across various domains, including education, content creation, and entertainment.

Outcomes: Hosted Web Application Platform and GitHub repository for local installation

Project Boundary: The application is supposed to be used only for media files with audio. It cannot interpret visual data in the file.

Project Timeline



System Architecture:

Hosting:

- App Hosting: Streamlit community cloud server (<https://streamlit.io/cloud>)
- Website Hosting: Infinity free website hosting (<https://www.infinityfree.com/>)

Models:

- LLM models: Llama3-8b-8192 via Groq API and Gemini-1.5-flash via Google API
- Speech-to-Text: Whisper-large-v3 via Groq API

Software Dependencies:

- Python v3.10.12 (for backend and frontend)

- ffmpeg v4.4.2 (used by pydub python library)

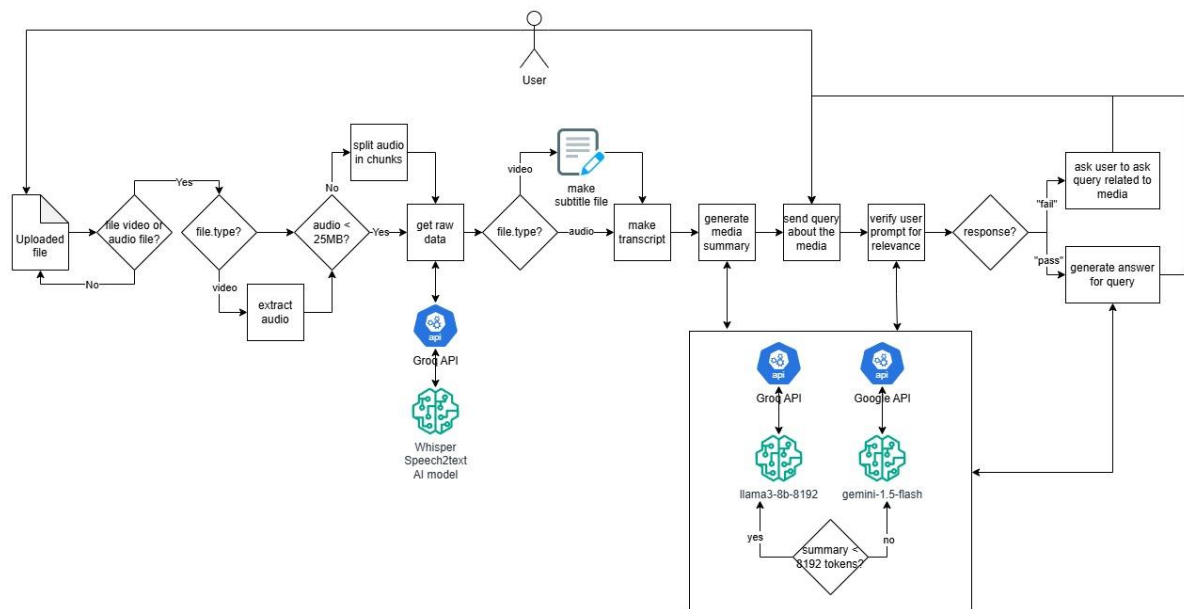
Python Dependencies:

- streamlit v1.36.0 (for GUI)
- groq v0.9.0 (for Groq API call)
- google-generativeai v0.7.2 (for Google API call)
- moviepy v1.0.3 (for extracting audio from video)
- pydub v0.25.1 (for splitting audio into chunks)

Codebase:

1. **main.py:** Acts as the main entry point for the Streamlit application.
 - Configures the Streamlit page.
 - Provides instructions for using the application.
 - Handles file uploads.
 - Integrates functionalities from other modules (response.py, summary.py, transcribe.py).
 - Manages the display and interaction with the user interface, including media players and chat responses.
2. **raw_data.py:** Handles the generation of raw data from media files.
 - generate_raw(file, filetype): Processes the uploaded media file and extracts raw data, which includes audio segments, duration, and language detection.
3. **response.py:** Generates chat responses based on user queries about the media content.
 - generate_chat_response(prompt): Takes a user prompt and generates an appropriate response related to the media content.
4. **summary.py:** Generates summaries of the uploaded media content.
 - generate_summary(file, filetype): Summarizes the content of the uploaded media file, providing a concise overview. It integrates functionality from response.py to generate the summary.
5. **transcribe.py:** Handles transcription and subtitle file generation for the uploaded media.
 - generate_transcript(file, filetype): Produces a transcript of the media content.
 - generate_subtitle_file(file, filetype): Creates subtitle files (in WebVTT format) from the media content.

System Workflow



1. **User Uploads Media:** The user uploads an audio or video file through the Streamlit interface.
2. **Processing:**
 - The *main.py* script calls functions from *raw_data.py* to extract raw data from the media.
 - *transcribe.py* is used to generate transcripts and subtitle files.
 - *summary.py* provides a summary of the media content.
 - *response.py* generates responses to user queries about the media.
3. **User Interaction:** The application displays the media player, transcript, and summary. Users can ask questions about the media content, and the application responds using the *generate_chat_response* function.

Methods

The project uses modern software development practices through its modular programming, iterative increment approach, continuous integration and continuous deployment (CI/CD) facilitated by Streamlit, and advanced prompt engineering techniques.

1. Modular Programming

The project is architected using modular programming, a design paradigm that breaks down the application into distinct, self-contained units, each responsible for a specific functionality. This is evident through the separation of tasks into different Python modules. The modular approach enhances maintainability, readability, and reusability, allowing developers to update or replace individual components without affecting the overall system.

2. Iterative Increment Approach

The development of MediaSleuth360 followed an iterative increment approach, a methodology that emphasizes building the software in small, manageable increments, with each iteration adding new functionality or improving existing features. This approach involves:

- **Planning:** Identifying and prioritizing features and tasks for each iteration.
- **Development:** Implementing the planned features in small, focused increments.
- **Testing:** Continuously testing the application to ensure each increment integrates smoothly with the existing system.
- **Review and Feedback:** Gathering feedback from stakeholders and users to guide subsequent iterations.

By iterating in this manner, the project benefits from regular feedback, which helps refine features and ensures the final product meets user expectations. This method also allows for early detection of issues, making them easier to address.

3. Continuous Integration and Continuous Deployment (CI/CD)

The project inherently supports CI/CD due to its development with Streamlit, a framework simplifying web application deployment. Developers can push changes to the GitHub repository, and Streamlit handles the deployment, ensuring that users interact with the latest version of the application. It reduces the overhead of manual deployment processes and allows for rapid iteration.

4. Prompt Engineering Techniques

Advanced prompt engineering techniques are employed to enhance the effectiveness of the Generative AI (GenAI) applications within MediaSleuth360. Key strategies include:

- **Persona Adoption:** Crafting prompts that adopt a specific persona or context to guide the AI model. It can improve the relevance and accuracy of responses.
Example: "You are an expert media analyst." or "You are a proofreader for prompts given by the user."
- **Task Specification:** The tasks are clearly defined for each role. This helps in focusing the AI's output on the required action.
Example: "Your task is to create a structured summary of the {filetype}." or "Your task is to provide relevant and concise information with timestamps."
- **Context Setting:** It ensures the responses are relevant and grounded in the specific content of the file.
Example: "Verify if the user query/prompt is relevant to the {filetype} context."
- **Instruction Emphasis:** Important instructions are reiterated to ensure the AI strictly follows them.
Example: "Strictly adhere to the context and do not include any additional information or commentary."
- **Verify GPT:** Using a secondary AI assistant to evaluate and verify the relevance of the user prompt. This approach ensures that the response presented meets a certain standard.

These prompt engineering techniques help refine the interactions between the user and the AI, making the responses more accurate, relevant, and aligned with user expectations.

Team Chart

Team Member	Role	Key Contributions
Krishirajsinh Puwar	Developer & Project Manager	• Project Ideation • Planning and Team Management • Developed app and the Website • GitHub & Code documentation • Prompt engineering
Manjunath Naidugari	Developer & Tester	• Optimized GUI and code base • App testing • Conducted User Acceptance Testing
Supratik Patel	Tester	• Developed an early-stage prototype • Improved system prompt for a better summary • App testing • Created project PowerPoint presentation

GitHub Link: <https://github.com/krishirajsinh-p/MediaSleuth360>

Current Progress and Future Plans

Current Progress:

- Completed the development of the core features:
 - multiple media support
 - time stamping
 - subtitle generation
 - summarization
 - interactive chatbot
 - media navigation
 - multilingual media analysis
- Implemented secure file upload and processing
- Handled larger media files, overcoming the Whisper model's limitations
- Implemented caching for performance
- Implemented graphical user interface for desktop
- Conducted preliminary user testing with a small group

Future Plans:

- Improve support for media inputs greater than 200MB.
- Expand capabilities to support chat conversations in multiple languages with consistency in responses.
- Add a feature to support conversational memory.
- Add features for teams to collaboratively analyze and discuss media content within the platform.
- Users can create custom personas and temperature controls for their domains or content types.
- Ensure complete security to prevent jailbreak, according to OWASP, and make improvements based on user feedback.